

Báo cáo bài tập EX7: Bài toán phân loại sử dụng SVM

```
# Chú ý: có thể biến đổi dữ liệu về dạng Tf-Idf trực tiếp sử dụng TfidfVectorizer()
# Bài tập:
# - thực hiện điều đó
# - hiển thị 10 từ trong văn bản đầu tiên có giá trị tfidf cao nhất
# code
import pandas as pd
modul_tfidf=TfidfVectorizer(stop_words=stopwords)
data_pre_tfidf=modul_tfidf.fit_transform(data_train.data, data_train.target)
first_vector = data_pre_tfidf[0].toarray()[0]

vocab = modul_tfidf.get_feature_names_out()
df_results = pd.DataFrame({'Từ': vocab, 'TF-IDF': first_vector})
sort_tfidf = df_results.sort_values(by="TF-IDF", ascending=False)

print(sort_tfidf.head(10))
```

	Từ	TF-IDF
17584	sách	0.534111
9552	học_sinh	0.263443
20254	tuyệt_đối	0.219063
15666	phụ_huynh	0.192140
14063	nhà_trường	0.184345
7816	gia_lai	0.174008
19345	tiếp_thị	0.165433
17869	sở	0.162776
7955	giáo_dục	0.153805
18130	tham_khảo	0.139133

```
# code
print("training")
modelx=svm.SVC(kernel='rbf', C=1.0, gamma='scale')
modelx.fit(X_train, y_train)
print("training completed")

print("testing")
y_test_pred_x = modelx.predict(X_test)
y_train_pred_x=modelx.predict(X_train)
print('Acc test= {}'.format(accuracy_score(y_test, y_test_pred_x)))
print('Acc train= {}'.format(accuracy_score(y_train, y_train_pred_x)))
```

```
training
training completed
testing
Acc test= 0.7797356828193832
Acc train= 0.998898678414097
```

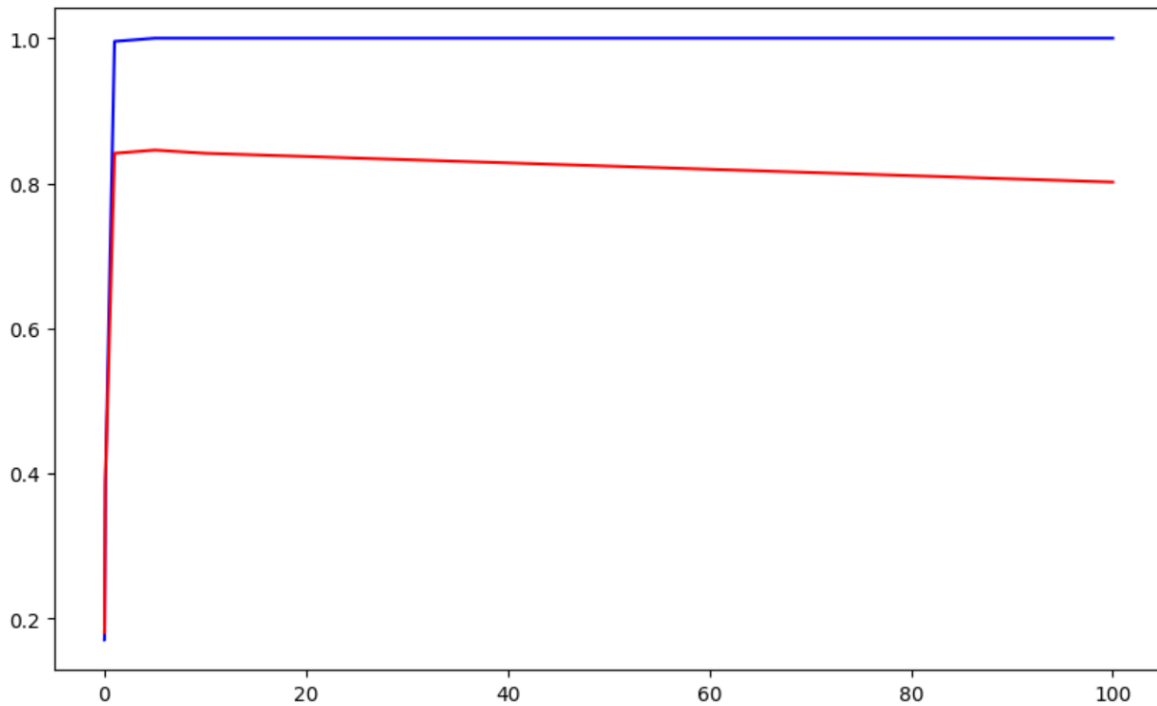
Bài 3: dự đoán nhãn của văn bản trên

```
# code
label_id=model.predict(input_data_preprocessed)
print(data_train.target_names[label_id[0]])
```

Thế thao

Bài 4: Vẽ Learning curve khảo sát Acc của SVM-linear với tham số C thay đổi

```
# code
# tham khảo miền của C
list_C = [0.001, 0.01, 0.1, 1, 5.0, 10.0, 100]
import matplotlib.pyplot as plt
import seaborn as sns
acc_train=[]
acc_test=[]
for c in list_C:
    modul=svm.SVC(kernel='linear', C=c)
    modul.fit(X_train, y_train)
    y_pred_train=modul.predict(X_train)
    y_pred_test=modul.predict(X_test)
    acc_train.append(accuracy_score(y_train, y_pred_train))
    acc_test.append(accuracy_score(y_test, y_pred_test))
plt.figure(figsize=(10, 6))
plt.plot(list_C, acc_train, color='blue')
plt.plot(list_C, acc_test, color='red')
plt.show()
```



Bài 5: Sử dụng GridSearchCV để tìm bộ tham số tốt nhất

```
: # code
# Có thể tham khảo giá trị các tham số như sau
params_grid = {'C': [0.001, 0.01, 0.1, 1, 10, 100],
               'gamma': [0.0001, 0.001, 0.01, 0.1],
               'kernel': ['linear', 'rbf', 'poly'] }

modul=svm.SVC()
grid_search=GridSearchCV(estimator=modul, param_grid=params_grid, n_jobs=-1)
grid_search.fit(X_train, y_train)
print(grid_search.best_params_)
print(grid_search.best_score_)

{'C': 100, 'gamma': 0.01, 'kernel': 'rbf'}
0.8490984153967579
```

Bài 6: phân loại với dữ liệu trên

```
##### exercise #####
# Yêu cầu: Ứng dụng mô hình svm vào bài toán phân Loại ảnh
# Gợi ý: dữ liệu đã được chia train, test, Áp dụng phần 2. và 3. để training và testing model. Chú ý nên có thêm phần tuning model
#####

# model = None
model=svm.SVC(kernel='linear', C=1)
datax=model.fit(X_train, y_train)

trainpredx=datax.predict(X_train)
testpredx=datax.predict(X_test)
print(accuracy_score(y_train, trainpredx))
print(accuracy_score(y_test, testpredx))
#####

1.0
0.9777777777777777
```